

AI in the SDLC: Meeting Transcript

Ascension x Insight Global · Operationalizing AI across the software delivery lifecycle

Participants

Speaker	Name	Context
Speaker 1	Marissa	Insight Global
Speaker 2	Erin	Insight Global (led the discussion)
Speaker 3	Jose	Ascension, Senior Director (digital engagement)
Speaker 4	Sandeep	Insight Global (presented the solution)

Speaker labels from the original transcript have been replaced with participant names. Consecutive lines from the same speaker are grouped into one turn.

Transcript

Marissa

Erin has been doing some of this work in one of your healthcare clients. Um, I will say things that—yes, go ahead.

Erin

So we don't—I have not seen—we've talked to a lot of providers about... No one, none of our direct customers, are doing it in a meaningful way. There are plenty that are saying, "Yeah, we're using AI." They're not. I mean, they're not—they're doing it in pockets. You know, they have some engineers doing some vibe coding, and they're like, "We're AI-enabled," and it's like, not really. The payer space is much further progressed. So we've seen Aetna Tech make some really good progress around identifying their whole solution. We are pretty involved in their digital space in this capacity, and then Optum, in pockets, has made some decent progress. But I do just want to be really honest with you, I have not seen yet a provider environment that has really actually done this in a meaningful way. So, just want to be really honest with you.

Marissa

The iceberg.

Erin

Uh, but we have seen other industries do this in a really meaningful way—other regulated industries: financial services, large banks. Bank of America, who we work really closely with, they have made tremendous progress. So it isn't "same-same," but that's probably—I mean, it is because healthcare is not adopting as fast as other industries. So I mean, you guys do it great, you will be one of the first, but there's not a ton of examples. There's a lot of customers that say, "We have identified our SDLC," but then when you actually look at it, no, no, you haven't—not really. You know, you're using some QA or some AI to help with QA, or like I was saying from a coding perspective, but we're not talking full delivery lifecycle. So, I just want to be honest about that.

Jose

No, I appreciate that. So awesome. I think that what you mentioned as the items that we're going to cover are good. A couple of things that I want to make sure we double down on—and Marissa, we chatted a little bit about this—so key components are: What's your perspective on operationalizing, if that is a word, AI in the software development lifecycle? So you've seen varying degrees of success, which is great. Industry-wise, healthcare is probably not a good benchmark to determine whether something is feasible or not. But I don't think that's a limit for us. I do want to clarify that I'm not looking at understanding this too much on the clinical side; this is more the digital engagement one.

Erin

Right.

Marissa

Yep.

Jose

Other pieces that are important to me: You know, when I talk about bottlenecks, I'm not really interested in moving bottlenecks from one place to another. It's more about how do we change the process and think about a delivery model that is more forward-looking, as opposed to what you've seen otherwise.

Erin

Yep.

Jose

Then the next piece is—and Marissa, I don't know if you had a chance to relay this—let's say that we have a team building agents or applications that have agents behind the scenes on a local device, and then they want to move them to a cloud environment, maybe that's Google, maybe that's Microsoft, Copilot, Cloud Run, whatever it is. What does that take? That's going to be important to me. Is it feasible? Can we do that? In the last few days, the next step that I want to explore—and you probably won't have that at this point—if I told you, "Hey, I have this developer that built this fantastic agent on their local device, and I want to move it to the cloud so I can have a cloud agent," what do I need to do that? What skillset do I need to make that happen? Because I know I don't have that skillset today on my team. So I do want to spend some time on that, and even if we don't have that right now, that would be fantastic after this meeting. Because I want to get your perspective, and I think if there's anything that we could explore beyond "Hey, how do we implement this?", it's "How do we get what's in people's heads and on their local machine into something that is usable across the board?" What do I need to make that happen?

Erin

Okay, so I think we're going to address a decent amount of that. To get down to the specific level of how to move agents off of local environments into cloud environments, we didn't go down to that level of specificity. But the content that Sandeep has been heroic in putting together and getting everybody's thoughts and feedback into a concise view—yes, it might take us the full hour. We're going to go deep; this is some dense content that we're going to review with you right now. So, I think somebody already hit record. Okay, cool.

Jose

The one thing that I will ask you, too, is as I told you what I want, what I do not want is a repeat from the Copado conversation, which is, "Hey, we just want to sell you a new application." Like, if that is what you're proposing, I will cut the meeting short. I want to understand a broader picture, more like, "Hey, this is what we can do," and then get into the details. I'm not asking you to give me work for free, I get that. But I do not want a massive sales pitch about the latest and greatest application that you all can build.

Marissa

Yeah, I think the only thing I'll just add on to that though, Jose, is as we get deeper into the conversation, if we feel really clear on our side that introducing a new technology is the right path with return on investment, obviously I want to feel like we can bring that up to you and not have it taken as like we're trying to shove something down your throat.

Jose

Yes, we can cover that, but I don't want to spend the bulk of the meeting—I want to be very transparent.

Marissa

Okay, no, addressed in that. Thank you.

Erin

Okay, Sandeep, do you want to share your screen? And really, this is all framed up in a knowledge graph, walking through full workflow top to bottom. Can you flip it to presentation mode, Sandeep?

Sandeep

One second, yeah, just a sec.

Erin

You know what I'm thinking? Let me save this down as a PDF, and Jose, I'm going to email it to you right now. This isn't a simple workflow, so I think we did the best that we could getting it into the most consumable version. But let me save this and send it to you so if you want to open it up on your screen... I'm the type of person that likes to print things out still, which I know our team hates.

Jose

In my notebook, I'm still writing stuff down. I'm trying to use the digital still, but I still need to see something that looks like paper, so I'm with you.

Sandeep

Why is that not showing in presentation mode?

Erin

No, you're still on your other screen.

Sandeep

I know, one second. Let me do this.

Erin

You want me to pull it up, Sandeep? I think I have the same version.

Sandeep

I think I got it, just a second. I'm putting it in presentation mode now.

Erin

Oh, there you go, there you go. Delayed.

Sandeep

Okay, perfect. So again, Jose, the idea of what we are trying to think about is... we are talking about this end-to-end SDLC. For that to be successful, currently, I know you guys are already integrated with Codex, right? We have never gotten the access to see how Codex is working, how the development phase is going on, or how much developers are using Codex right now. But there are some limits from the token wise and also the context that Codex is giving for us to make sure we trust it 100% to deploy it into production levels, or to use it end-to-end per se, right? So what we are trying to see is—because right now we have multiple sources for authorization systems like Jira... We are using Jira, GitHub, and Confluence, where we are updating a lot of data, and your entire code repository is right now in GitHub and stuff, right? So whenever we are trying to talk to Codex, most of the stuff it's going to do a keyword search or a RAG search, where you get a lot of information, and we have to choose what information to be used for us to make sure we are getting the right context of it. So for us to go forward and make sure that we fully trust our AI system and also make sure we can use it for the deployments into our dev environments, and from there into our higher environments, we need the right context of what we are asking. So the layer that we are trying to solve is the context layer. Are we giving the right context? Are we getting the right context out of what we are asking from our current LLM? Not just using LLM, but also using our knowledge database that we are going to create. I'm not taking a physical database, but I'm talking about our entire knowledge. Are we saving it in a form where it understands the relation between everything? Let's say we talk about HFNs, right? How are HFNs created? What is the relationship between HFN account, doctor, practitioner, and everything, right? Are we saving it in such a way that context is going to be 100% right for the LLM? If context is right, I'm getting the right output from the context. How can I integrate it to my systems where I can make sure I develop it using AI and deploy it using AI, right? So that's for the system, but this slide shows how we can achieve the context. Layer 1 is our source systems: Jira, Confluence, or whatever source system that we want to use as authoritative sources. Jira, Confluence right now, Salesforce, Salesforce DB, GitHub, Copilot, Google Drive. Layer 2 to Layer 5 is all in-house, in-memory—what we are trying to use is all going to be the code that we are going to develop. Layer 2 is nothing but the ingestion. How are we going to connect Jira, Confluence, Salesforce, GitHub, using the connectivity layer into the ingester layer? Here we get all the information from Jira, Confluence, Salesforce into multiple different tables. Again, this is all code, it's not going to be visible. Everything that we are writing is going to be in-memory in Java, Python, or anything. For Salesforce, we can also figure out if we can do that in Apex. But everything will be written in the code, so it will get the tables and the data required specific to Jira, specific to Confluence. These are individual stuff that it's going to build. And the fourth layer is where it makes that universal data—having complete universal information about everything that we are talking about. Again, this is also in-memory. The output of Layer 1 to Layer 4 is Layer 5. We are going to build one bigger JSON with object levels: Object 1, Object 2, and Object 3. It will be the bigger JSON file with everything that we have information on in our systems. This output will be stored—let's say for testing purpose right now, what we are thinking is let's save it in a file in Salesforce. But when we are going live or when we really want to implement, there will be a completely different solution. We want to use Google Cloud or something because we don't have it right now. So let's say we are going to store this in our Salesforce file somewhere. The main part for the context is the graphical layer. Graph database is what, in Google, they call Spanner. There are multiple graph DBs available in AWS, Azure, Google, and there are other open sources that we can also use, like Semantica AI. Semantica is one graph DB where whenever we give a JSON, it will understand the relationship and build the graph relationship for each and every piece of information that is there in the JSON. It slices that information and stores it in slices. That's the context layer that it is going to be stored in. Right? That context can be integrated into anything. You can integrate it to your agents, your current Codex, Salesforce—anything we can integrate it to. When we are

integrating it to Codex or any agents, the moment that we search, we will make sure that the first request goes to our knowledge database, which is our graph DB, and it will bring the information from that DB. It's not going to go and talk to an LLM, it's not going to consume any tokens, nothing, and you will get the right context mainly because we are storing it in the relation. It's called edge and node relation. Node is nothing but the object or the thing; edge is nothing but the relation. So it will have the proper relation and get you the right context. When you see the right context and when we are able to fine-tune it to the level of every time that we ask—we have to do some fine-tuning in the graphical layer—once we have that fine-tuned, you'll get the right context when you ask a question. That right context can be integrated into the development layer. How is that going to work? Let's say a product manager is trying to update a new requirement into Jira, right? He will be interacting with this, he will create the story. In the story, it will give the right context of what needs to be updated: what Apex class needs to be updated, where it needs to be updated, what test classes need to be run, and everything. And that will be pushed from this layer into Jira. Think of it like Jira will be in an open stage when the product manager sends it to Jira. If your developer or SCM reviews it and changes the status to "In Progress", that will automatically push that code into your dev environment where we can test it, right? And from there on, human intervention will be there to push it into the next layers. But for basic developments, if we are really confident when we achieve that level of confidence on the context that we are getting, we can use an automated SDLC kind of workflow. You can push it into Jira, and from Jira, it can develop and push it into dev. From there we test it, review it, and then we can add human intervention to move forward into upper layers. But for everything to work, what we are trying to do is make sure our authorization systems have connectivities or ingestions, convert them into a full-length JSON, connect the JSON to your graph DB, and have the graph DB understand the relations and build them in the graphical layer so that you can connect it to any agents—Codex or anything that you want. From there, we can query and ask questions. If you are connecting it to Codex, we'll make sure Codex first goes and refers to the graph DB to get the right context. Again, the JSON will keep updating itself; we can schedule how frequently it updates so all the information will be accurate. Whenever you make changes, it will keep updating the JSON file accordingly, the graph DB will be updated, and you will get the right context out of it. What we are trying to do is integrate it to the next level, which is Codex or whatever agents, to the final output. We'll connect Jira, and from Jira to Salesforce dev environments and stuff, making sure whatever context—if you are agreeing to that—gets pushed into Jira. If it's in Jira, SCM will review it without having to manually code it, change the status, and it will be pushed into your dev environment to look at what the changes are. If we approve it, then from there on human intervention will be there, and we can push into the next layers. This framework up to the graph DB, we already built it right now; the framework is available if you want to look at a POC. It's a generic framework, but we can give a demo—not in your Salesforce environment, but we built it in a normal environment, so we can give a high-level demo there. Anything can be changed here. Till Layer 5, it's all code. From there, what you want to use as a graph DB—if you want to use Google or an open-source option for testing—we can decide from Layer 6, Layer 7, and Layer 8. But till Layer 5, it's all in the code. Looks like our IG Labs already built that framework. If we want to look at a casual demo of what it is in our system, we can showcase that.

Erin

Before we go into that, let's pause for a second. Jose, I want to make sure conceptually, at a high level, you're on board with what we're presenting here. Essentially taking all of our knowledge feeds across these layers into a singular knowledge graph. Then Codex queries against that graph instead of trying to dig through an entire application for answers. In this model, Codex will never invent a connection that's not actually there. Your team members still own final approvals on everything, too. Eventually, maybe in a future state, technology will get to a place—and our guardrails, evaluation criteria, and governance will get to a place—where things can auto-publish or get pushed to production. But at a high level, that's not what we're talking about as step one in terms of how you operationalize first.

Sandeep

We are building that pipeline for the AI, and we are doing it for multiple others—like J&J, we were just talking about the same solution, building the pipeline for their AI layer, the context pipeline. Right now, I'm building the context and the pipeline for the AI to give the right outputs and trust it more than what it currently is. Every time I search for a file, let's say in Copilot for my organization, it brings me 10 files with a keyword, and I need to search for what file it actually is. Now, when I ask our knowledge graph where it's saved, what Apex class it has been using, everything from the coding side—since it's integrated into the Git repository—it will give you the exact file we need to change, where we need to change it, and it can also estimate effort. Because the graph DB is an intelligent layer, it has its own intelligence to tell you how much time it will take. To get this story done, a product manager has to first review it and ensure he pushes it into Jira. Again, that solution will be reviewed, and we can push it into the environment. So, no manual intervention or coding up to the dev layer is what we are looking at. But if we trust it more, we can move it to the next layers.

Erin

What do you think so far, before we keep rambling?

Jose

Yeah, I think this helps from a behind-the-scenes perspective, and I like how you're laying it out in different layers and how you get it all compiled in a single layer for retrieval and changes. When you take this—let's say that we put this together, someone else puts it together, and we have this fantastic setup where Codex connects to something behind the scenes, let's call it a database for simplicity. How does this change my delivery model? This is a technical tech stack with connectors, which makes sense. I still need a PM to put a PRD together. I still need an architect to put an HLD together. I still need the developer to do the coding or configuration changes, and I still need the QA team to test whatever output you put in there. Now, in this scenario, let's say each one of those individuals has Codex today connected to this code database. They will ask Codex, "Hey, put the PRD together, put this thing together." I still have a sequential process where I require certain capabilities or skillsets from individuals—or staff, for what it's worth—to deliver those different outputs. How do I change that? Because what I'm doing in that case is increasing efficiencies within a specific practice. I increase efficiency in PRD creation, HLD, development, and QA; I create individual efficiencies in different pockets. That maybe increases my overall efficiency 20–30% because I increase them all at the same time, but it doesn't really disrupt or change... I still have the same basic sequential model where the handoff still exists. How can I use this to change that?

Sandeep

Right now, you guys are moving into one-week sprints anyway, right Jose? So you're already planning to do more work than we are doing right now.

Jose

Ignore the changes that we're making. Call it a three-day sprint, it doesn't matter for what it's worth. I'm thinking bigger, I'm thinking more. These are pilots; these are things that we're just testing out.

Sandeep

Sure. From an efficiency point of view right now, let's say we are talking about the developer and SCM. We are not doing anything manually to develop code; you need somebody to understand what this context is giving. Again, from a PM point of view, do we need interaction between business and our engineering team? That's what you need to decide. For how we operate, I feel my SCM understands the requirements better than a PM because he is the one developing a lot of stuff. Do we have that talent right now who can go to business and understand requirements? We have PMs mainly to understand functional requirements, but your SCM also knows functional requirements end-to-end. You need somebody who understands what the output is and how it impacts things. One person needs to decide if this is better to go into dev or not. You don't need two developers to look at this and separate stories to develop by hand. You will get the code, it will be pushed into your dev environment, and it will tell you what needs to be tested. If we want to automate to the level that it tests automatically and gives results, we can do that. But in the initial phase, we have to be 100% sure this model is working. For that, you may need a few team members, like one QA, to make sure the output is right. But thinking about devs and SCMs, we don't need the dev layer completely there.

Erin

Sandeep, let me add one more thing to address directly what you said, Jose. Honest answer: What we're showing right here, the delivery model really is not changing significantly. Think of this as an elevation, not a replacement. However, I would caution you that in an organization, to jump from where you are today to full operational organizational change management uplift, we would not recommend jumping straight there. This is the sequential next step. If you want us to take the vision further—let's say this is successful, how do we eventually... what does future state look like operationally down the line?—we can put more thought into that. But I wouldn't want to jump to that crazy change given the current state.

Jose

I agree, I wouldn't do that either. But here's how I look at it, Erin. Imagine I want to go from St. Louis to Chicago, right? And I'm going to take the train. I know that I'm in St. Louis and that I want to get to Chicago. A hundred years ago, it might have been difficult to think, "Hey, I want to get to Chicago using this thing." These are different stops. What if I tell you I don't even know where I'm going right now with this? You're telling me what the next station is. I don't even know whether I'm going to Chicago, Kansas City, or New York via the train. This is telling me, "This is your next step," and I think that's great. There is a lot of value in there. I don't think we can disrupt an organization and change operations overnight—there's change management, a lot of things. But I want to know what my end state is, right? That's where I want to think. It might sound unrealistic, and maybe we'll figure that out, but I want to know.

Erin

You just tapped right into my brain! I was wondering how many future states we should model out potentially. Here's the challenge that we face: As I was saying before, there are really very few organizations that have done this in earnest. To vision it out... do we want to spend time doing that if there's a good chance none of that is actually going to be realized, or at least not anytime soon?

Jose

Help me understand, why not?

Erin

Okay, we can. I just want to know if that's where you want to spend mental cycles versus what is actually the next station stop.

Jose

If I think about it from a delivery perspective, Erin, it's not necessarily a waste of time. It is partially a waste of time, but let's look at how AI companies are delivering software today, right? They might have an architect working as a PM, who is working as a developer in a way. They have agents deployed that are capable of doing a lot of the work. Let's remove the Salesforce component out of the equation and say we're talking about custom development entirely. Then we say, "I have my set of agents that are developers, my set of agents that are architects, my set of agents that are QA," and I become the orchestrator of these agents. This is an agent environment where I can tell an architect agent, "Hey, go ahead and put this thing together." Again, it's not that I want to think about replacing staff; it's about how I enhance the capabilities that my staff have. I'll still have an architect, a developer, and QA people who are going to review that. Maybe, maybe not, I don't know. But I can say, "Given my company context that I can feed..." Imagine this, you put it in a broader perspective: I feed it what we have built in Google Cloud, what we have built in Salesforce, and a bunch of other things. I tell my architect agent, "Hey, I need you to think about an application that will replace all the EHR portals that I have in my organization. Put an HLD together based on the context that you know about my company." You know that I have 3,000 applications at Ascension, and you know roughly what this is. Again, you need to feed all this information there. Maybe I'm using Claude instead of Codex because I think that service is significantly better for planning. It will put something together. Whatever output I generate from there, I can check with Codex, OpenAI, or other tools to double-check if the output is correct. That type of thing changes the structure and model for how we deliver, right?

Erin

100%, yeah.

Jose

To me, what I think about is: Is there a way to deliver software that gets closer to how OpenAI, Anthropic, or Facebook delivers software? Facebook tried to deliver software a month ago, miserably failed, didn't get rid of 20% of their staff, and ended up with 10%. I don't want to do that. But there are companies trying to change; they're pushing the limit.

Erin

100%, 100%. Yeah, I hear you. We can put more thought against that. Transparently, can I speak plain?

Jose

No offense, but you can even stop the recording if we need to.

Erin

No offense, but Ascension is not the customer where in my mind I'm like, "Let's vision it to the future and beyond." In what we're showing—and we can send you a detailed slide that breaks down each layer further, Sandeep can pull those up if we want to go there... For this first pass, we really wanted to think about transformation happening as an elevation, not a replacement or fundamental change of operational processes where you still need the same people—probably less of them in time, and probably not that much time either. But what we're looking at first is shifting developer time. Instead of tracing dependencies and writing boilerplate Apex and tests, we're shifting that focus toward generated change plans and owning design decisions. Your tech leads and architects gain a more structured approval from this.

Jose

Yes, there is value. Here's the thing, Erin. Maybe I'm misstating what I'm trying to say. I think there's legitimate value in what you're presenting from an efficiency gain perspective across discrete components of my delivery model. That is 100% true. But I'm going to flip the question back at you. Okay, I am a senior director asking you how to change my delivery model. How many senior directors in Ascension are right now talking to a gazillion vendors trying to figure out the exact same thing? How is proposing this thing not the exact same thing everyone else is getting from other vendors, where we're all coming up with the exact same strategy that doesn't fundamentally change our delivery model? I agree Ascension is not the starting line for operational change. Healthcare in general isn't. Removing my healthcare hat for a moment—and I know it's an Adidas one, but removing the healthcare hat for a moment—I'm putting on my consulting hat, the one I had before I joined Ascension. If I really want to change something, I have to propose something that sounds somewhat outrageous for the organization I'm in, but is still legitimate. How do I push the envelope a little bit more? Because all I'm doing is competing with folks presenting the exact same thing—just change the name from Salesforce to Adobe. "How am I going to change the delivery model for Adobe?" Okay, you have that efficiency, but I still have the exact same expense from a staff perspective. I've increased operational expenses because I have more tools to maintain in the ecosystem, plus breaking points in custom code. Set that aside, we're all presenting the exact same thing. So to me, I must be a really nice presenter, people must really like me, for them to say, "You have a stellar option to fix this." And I don't think I'm that special. I don't want to just compete with the same... I'm just another soldier in here, and I want to be the general.

Erin

Thank you very much, thank you very much. This is so helpful to have that context, because what I'm hearing is: for you to get buy-in to make change, you almost need to go pie-in-the-sky and paint the whole picture. Sometimes I waver because I don't want to be the partner that paints this beautiful future knowing a lot has to go right to get there.

Jose

I will die on the hill of a proposal being so outrageous and unrealistic that we might not materialize it, rather than die in the valley where everyone else is dying with the exact same idea. Selfish as it sounds, I want to have the best idea only for me, and that's the one I want to present.

Erin

Okay, so being mindful of time, we could stay on and walk through layer-by-layer, but you get it at a high level.

Jose

Yes.

Erin

I sent you the high-level summary, but Sandeep can share the entire slide deck. Let us do what you said: paint the picture. You've basically taken my handcuffs off and said, "Don't live in this box, go think big." Show me the art of the possible. My first pass was to give something realistic. I've met with plenty of customers where I've gotten the feedback, "Great, this is so far beyond what we're doing. You are out of touch, sis, bring it back to reality." But if you want to open the window and let me fly out into the universe, I will be happy to.

Jose

Here, I'm going to give you a couple of examples. There are a few URLs. When I look at a delivery model from things that have been published, that is public knowledge; I can figure out how to deploy agents in an environment and do all these things. None of these things are going to be materially different from what everyone else is doing. What I want to think about is: Here's what all of these companies have done, this is the model they tried—maybe they failed, maybe they didn't—and then let's add the flavor of engagement services on top of that. Or more—maybe they want more than engagement services, we'll see. But what I need is something that will materially change the way we deliver. Maybe the answer is you'll never get there, or maybe I have to go somewhere else that does something better or faster, but at least I want to try.

Erin

Okay, now can I ask a question back to you? Take your mental handcuffs off too. You say we're materially changing the way that we deliver. Fast-forward a year or two from now, and that's been done successfully. What becomes your measure of success? Is it producing at the same rate with OpEx at a fraction of what it was, or does it go beyond that? Paint for me the ideal future state—not how you got there, but what that looks like and why it's your ideal.

Jose

At what level do you want to talk? Company level, team level, or context?

Erin

Start me at the team level, but how does that impact others?

Jose

Okay, let's think about the team level. How do I measure success from a team? Do I want to reduce OpEx? 100%. Does that mean I want to reduce staff? That's not what I'm aiming for. There may be reductions in capital expenditure; I may not have as many contracts and contractors. Do I want to spend less money with the tools I have? Sure, that's everyone's ideal. How do I deliver faster? How can I use the staff I have today and deliver much faster than today? I'm going to use the example of PDN because PDN is a good example. That team on its own is tied to a company goal. We could argue whether readmissions are going to be materially changed by me sending an SMS to someone telling them to take care of something. A cardiologist might disagree that you change readmissions based on a text message, but I'm still tied to a company goal with that. Right now, it takes me two weeks—what the team has told me—simply to create another SMS template and another branch in my journey in Marketing Cloud to deliver one SMS that is the exact same thing to another market. My challenge to you is: I should be able to do that in half a day. That opens the door that if you have an idea and say, "Hey, let's run the experiment, let's try a different type of SMS," I can do it fast, fail fast, and rule things out faster. Even with testing strategy, there's software I may need to do SMS testing. I have massive, untapped capabilities right now with AI to do better testing and fail faster internally, as opposed to failing faster by sending an SMS to a person the wrong way because the template created wasn't what it was supposed to be. Through the lens of purely delivery from a team perspective, that right there is a material change from where I'm at today. I struggle to justify a two-week cycle for the exact same thing, just with a different market name. It's insane, it does not make sense. It doesn't fit in anyone's mind that is not in the Salesforce space. This is insanity that it takes me three months just to do this. And I'm tied to company goals. By the way, we're behind on our goals because we're not delivering as fast as we could possibly deliver. I attribute it to the operations of our team and organization that are materially impacting our ability to do that faster. I can't continue business as usual because someone else will come in—Deloitte, BCG, McKinsey—who will come up with a fantastic idea. An executive will say "Yes," and in two weeks I lose my job because all I wanted to do was increase development delivery speed by 20%. That will not cut it. Fast forward to Edward Jones doing the exact same thing: "Let's reduce contractors and see how we land in a year," and they're just hiring everyone back. Maybe we land there, maybe we don't, but we have to do something.

Erin

Going back to your analogy: To travel to New York City, the way we're set up right now, you make three stops on a train or drive a car. I could put you on that same path in a train to get you there a little faster, or I could completely break the model, put you on an airplane, and get you there directly.

Jose

Right. What I'm competing with is that everyone is talking about keeping the same horse, maybe adding wheels in the back so the friction is less and the horse moves a little faster. It's the same thing—a different toilet. That's not going to mean anything. But from a team perspective, that's a pod. Move that to engagement services... I can go layer-by-layer to where I want to get, but does that help?

Erin

Totally, totally. I don't want to waste more time since we haven't visioned that far out. What I would ask is that we put thought into visioning how we get on an airplane and off the ground. But at the same time, I don't want to lose sight of efficiency pickups that can happen in the immediate future. Both things can be true at the same time.

Jose

I agree with you, you can do both at the same time. That will buy me time to put the grandiose vision in place. So by all means, there is a ton of value. I know I'm sometimes very difficult to please, but that's where my brain is at.

Erin

I'm having an epiphany moment where I connected and understand you better now, Jose, so thank you. When we follow up on asks, we need to do a better job on the front end—not just "tell me how to bring AI into our current workflow." We need to repivot the conversation to the end goal, showing the short-term and long-term gains.

Jose

Don't pigeonhole me into just being an engineering lead trying to increase efficiency on an engineering team using the exact same legacy model used for 50 years. That's something I need to do as part of my role, but I want more than that.

Erin

100%. This has been eye-opening and very helpful. We need a little bit more time, so probably not next week. We do have customers thinking bigger, so we can pull Sandeep, Nur, Vish, or Yash into the conversation and cross-compare what hyperscalers and big tech are doing. If you're saying, "Show me the art of the possible," then let's go.

Jose

Our executives are meeting with executives of those companies—Salesforce, OpenAI—and being told fantastic stories about technology. All they say is, "I want that." On our side, our job is to materialize that idea to an extent.

Erin

Honestly, it's so relatable. Our executive team at Insight Global wants that every single time. I've seen the downside of painting a pie-in-the-sky vision where executive expectations become that it's reality next quarter, which isn't feasible. So I won't be the partner saying pie-in-the-sky without also addressing the immediate.

Jose

Yes, I agree with you. Present it to me, convince my directors that that's the way we should operate, get them on board, and align them.

Erin

All right, we'll spend some time and give you an update next week on when you can expect more of that vision back from us.

Jose

Sounds good. I will take a look at this anyway because I think it's very valuable.

Erin

All right, thanks everyone. Bye.